

Assignment-3

1. Create a custom hook called `useSearchMovies(query)` that takes a search term as a parameter and fetches movie data from the OMDb API (http://www.omdbapi.com/?apikey=YOUR_API_KEY&s=QUERY). Return the fetched data from the hook.

Answer:-

`useSearchMovies.js`

```
import React, { useEffect, useState } from 'react'

const useSearchMovies = (query) => {

  const [movies , setMovies] = useState([]);

  useEffect(()=>{
    if(!query){ return ;}
    const fetchMovies = async ()=>{
      try{
        const response = await
fetch(`http://www.omdbapi.com/?apikey=YOUR_API_KEY&s=${query}`);

        const data = await response.json();
        setMovies(data.Search || []);
      }catch(error){
        console.log("Your are Fetching the error", error);
      }
    }
    fetchMovies();
  },[query])

  return movies;
}

export default useSearchMovies
```

`Movies.jsx`

```
import { useState } from 'react';
import useSearchMovies from './useSearchMovies';

const Movies = () => {
  const [search, setSearch]= useState("");
  const movies = useSearchMovies(search);
  return (
```

```

    <div>
      <input type="text"
        placeholder='search Movies'
        value= {search}
        onChange={(e)=>setSearch(e.target.value)}
      />

      {
        movies.map((movie)=>{
          return <div key={movie.imdbID}>
            <h3>{movie.title}</h3>
            <p>{movie.year}</p>
          </div>
        })
      }

    </div>
  )
}

export default Movies;

```

App.jsx

```

import React from 'react'
import Movies from './component/Movies'

const App = () => {
  return (
    <div>
      <Movies />
    </div>
  )
}

export default App

```

Question 2. Update your useSearchMovies hook to manage and return loading and error states along with the movie data. Display a loading spinner while fetching and show an error message if the API call fails.

Hint: Use useState to track loading and error, and update their values inside your useEffect.

Answer:-

useSearchMovies.js

```
import { useEffect, useState } from 'react';

const useSearchMovies = (query) => {

  const [movies, setMovies] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("")

  useEffect(()=>{
    const fetchData = async () =>{
      try{
        setLoading(true);
        setError("")

        const response = await
fetch("https://69f9ba30c509a40d3aa32d0f.mockapi.io/movieData")

        if(!response.ok){
          throw new Error("Failed to fetch Data....");
        }

        const data = await response.json();
        // console.log(data);

        const filterData = data.filter((movie)=>{
          return movie.title.toLowerCase().includes(query.toLowerCase());
        })

        setMovies(filterData);
      }
      catch(error){
        console.log("Error is:- ", error.message);
      }
      finally{
        setLoading(false);
      }
    }
    fetchData();
  },[query])
  return { movies, loading, error};
}

export default useSearchMovies;
```

Movies.jsx

```
import { useState } from 'react';
```

```

import useSearchMovies from "../component/useSearchMovies";

const Movies = () => {
  const [search, setSearch] = useState("");
  const { movies, loading, error } = useSearchMovies(search);
  return (
    <div>
      {console.log(movies)}
      <input type="text"
        placeholder='Serch your Movies'
        value={search}
        onChange={(e) => {
          setSearch(e.target.value);
        }}
      />
      {loading && <h2>Loading.....</h2>}
      {error && <h2 style={{color:"red"}}>{Error}</h2>}

      {movies.map((movie)=>{
        return <div key={movie.id}>
          <h2>{movie.title}</h2>
        </div>
      })}
    </div>
  )
}

export default Movies;

```

App.jsx

```

import Movies from '../component/Movies'

const App = () => {
  return (
    <div>
      <Movies />
    </div>
  )
}

export default App

```

3. Add a second `useEffect` inside your `useSearchMovies` hook that logs a message to the console every time the fetched movie data changes.

Hint: The second `useEffect` should depend on the movie data state.

Answer:-

```
import { useEffect, useState } from 'react';

const useSearchMovies = (query) => {

  const [movies, setMovies] = useState([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("")

  useEffect(()=>{
    const fetchData = async () =>{
      try{
        setLoading(true);
        setError("")

        const response = await
fetch("https://69f9ba30c509a40d3aa32d0f.mockapi.io/movieData")

        if(!response.ok){
          throw new Error("Failed to fetch Data....");
        }

        const data = await response.json();
        // console.log(data);

        const filterData = data.filter((movie)=>{
          return movie.title.toLowerCase().includes(query.toLowerCase());
        })

        setMovies(filterData);
      }
      catch(error){
        console.log("Error is:- ", error.message);
      }
      finally{
        setLoading(false);
      }
    }
    fetchData();
  },[query])

  useEffect(()=>{
```

```

        console.log("Movie Data Updated....", movies);
    }, [movies])
    return { movies, loading, error };
}

export default useSearchMovies;

```

4. Use ChatGPT or GitHub Copilot to suggest improvements for your useSearchMovies custom hook, focusing on better error handling or code readability. Implement at least one suggestion and briefly describe what you changed.

Answer:-

One improvement suggested by ChatGPT/Copilot is to **avoid making API calls when the search query is empty** and to **clear old movie data** when no search term is provided. This improves performance and prevents unnecessary network requests.

useSearchMovies.js

```

import { useState, useEffect } from "react";

const useSearchMovies = (query) => {
    const [movies, setMovies] = useState([]);
    const [loading, setLoading] = useState(false);
    const [error, setError] = useState("");

    useEffect(() => {
        if (!query.trim()) {
            setMovies([]);
            return;
        }

        const fetchMovies = async () => {
            try {
                setLoading(true);
                setError("");

                const response = await fetch(
                    "https://69f9ba30c509a40d3aa32d0f.mockapi.io/movieData"
                );

                if (!response.ok) {
                    throw new Error("Failed to fetch movies");
                }
            }

```

```
const data = await response.json();

const filteredMovies = data.filter((movie) =>
  movie.title
    .toLowerCase()
    .includes(query.toLowerCase())
);

setMovies(filteredMovies);
} catch (err) {
  setError(err.message);
} finally {
  setLoading(false);
}
};

fetchMovies();
}, [query]);

return { movies, loading, error };
};

export default useSearchMovies;
```